Predicting Stock Prices Using Deep Learning Techniques

Bazyl Sheikh, Arsalan Tahir, Usayd Arsalan
Department of Artificial Intelligence & Data-Science
FAST-NUCES, Karachi
k242532@nu.edu.pk, k242568@nu.edu.pk, k242553@nu.edu.pk

Abstract-This project uses Long Short-Term Memory (LSTM) neural networks to predict next-day stock prices for multiple technology stocks. Historical market data and technical indicators are processed, scaled, and used to train individual models per stock. Predictions are evaluated using Mean Absolute Error percentage and R^2 score, and results are presented through a web interface with visualizations and decision-support signals. The study demonstrates the potential of deep learning for financial forecasting while acknowledging limitations due to market volatility and uncertainty.

Introduction

Stock market prediction is a complex task due to the highly volatile, nonlinear, and time-dependent nature of financial data. Conventional statistical and machine learning techniques often struggle to capture long-term temporal patterns present in historical stock prices. In recent years, deep learning models have demonstrated strong capability in modeling sequential data, making them well-suited for financial time-series forecasting.

This paper explores the use of Long Short-Term Memory neural networks for next-day stock price prediction by analyzing historical market data combined with technical analysis features. Long Short-Term Memory networks are specifically designed to retain long-term dependencies in sequential data, allowing them to learn meaningful patterns from past stock price movements. We developed an artificial intelligence based stock prediction platform that supports multiple technology sector stocks, including Apple, Amazon, Google, Meta, Microsoft, and Tesla.

Our goal is to develop an accurate and scalable stock price prediction system that provides next-day price forecasts along with actionable trading recommendations. The system incorporates commonly used technical indicators such as moving averages over multiple time windows and the relative strength of price movements to enhance predictive performance. Real-time stock market data is retrieved from an online financial data service, and individual deep learning models are trained separately for each stock to better capture asset specific behavior. The predictions and recommendations are presented through an interactive web interface, making the system practical for real-world analysis and decision support.

Project Motivation:

The motivation behind this semester project is to explore and understand the application of deep learning techniques, particularly Long Short-Term Memory neural networks, in financial time-series prediction. From a learning perspective, the project provides hands-on experience with sequential data modeling, feature engineering using technical analysis, and training deep neural networks for real-world datasets. From a research-oriented standpoint, it allows the investigation of how Long Short-Term Memory models perform when trained separately on multiple stocks and enriched with technical indicators.

This project aims to bridge theoretical concepts studied in coursework with practical implementation by developing a stock price prediction system. By integrating data collection, model training, prediction, and visualization into a single platform, the project enhances conceptual understanding of deep learning workflows while demonstrating the potential and limitations of artificial intelligence techniques in stock market forecasting.

In addition, the project emphasizes the importance of building scalable and modular machine learning systems that can adapt to multiple financial assets. Training individual models for each stock enables a deeper analysis of asset specific behavior and highlights how different market characteristics influence model performance. This approach also supports comparative evaluation across multiple stocks, strengthening the analytical and research value of the project.

METHODOLOGY

In data science projects, a systematic and step-by-step process is essential to ensure reliable analysis and meaningful results. This project follows a structured data science workflow, beginning with problem definition and progressing through data handling, model development, and result presentation.

The methodology of this project is divided into the following six processes:

1. **Setting the Research Goal**
2. **Retrieving Data**
3. **Data Preparation**
4. **Data Exploration**
5. **Data Modeling**
6. **Presentation and Automation**

Setting the Research Goal:
The research goal of this project is to analyze the effectiveness of Long Short-Term Memory neural networks for next-day stock price prediction using historical financial time-series data. The objective is learning-focused and research-oriented, emphasizing model understanding, performance evaluation, and system design rather than providing financial or investment advice.

RetrievingData:
In this step, historical stock market data is retrieved programmatically to ensure consistency and reproducibility. The data is collected using the ***yfinance library***, which provides access to financial data from ***Yahoo Finance***. This library is used because it offers reliable historical and recent stock price data, supports multiple tickers, and allows automated data retrieval through a simple programming interface.

The dataset is collected over a predefined time range and serves as the primary input for further preprocessing and model training.

```
1. BEGIN
2.   SET start_date to predefined historical date
3.   SET end_date to current system date
4.
5.   FUNCTION load_data(stock_symbol):
6.           FETCH historical stock data for stock_symbol
7.           FROM online financial data source
8.           BETWEEN start_date and end_date
9.
10.          CONVERT date index into a column
11.          RETURN processed dataset
12. END
```

```python
START = "2014-01-01"
TODAY = date.today().strftime("%Y-%m-%d")

# Define a function to load the dataset

def load_data(ticker):
    data = yf.download(ticker, START, TODAY)
    data.reset_index(inplace=True)
    return data
```

DataPreparation
Data preparation ensures that raw stock data is cleaned, formatted, and structured for effective modeling. This process is further divided into three key steps: **data cleansing**, **data transformation**, and **combining datasets**.

DataCleansing
Data cleansing involves handling missing removing inconsistencies, and verifying the integrity of the data.

```python
data = yf.download('APPL',start=START,end=TODAY, auto_adjust=True)
df=data

#data.reset_index(inplace=True)
#print(data.shape)
print(len(data.columns))
print(len(data))
data.reset_index(inplace=True)

#handles multilevel ad flattens
if isinstance(data.columns, pd.MultiIndex):
    data.columns = [col[0] if col[0] != '' else col[1] for col in data.columns]

data.head()
```

Why we flatten multi-level columns? *Sometimes, when downloading stock data using libraries like **yfinance**, the resulting dataset may have multi-level column headers. For example, a single column like `Close` could appear as `('Close', '')`. This happens when the library returns extra metadata. Flattening multi-level columns*

i. Converts columns to a single level

ii. *Ensures consistent naming for further processing*

iii. *Prevents errors in feature engineering, model training, or data merging*

```
BEGIN
    SET start_date to "2014-01-01"
    SET end_date to today's date

    FETCH historical stock data for 'META' from Yahoo Finance
    STORE data in dataframe df

    PRINT number of columns in df
    PRINT number of rows in df

    RESET index of df to convert 'Date' from index to a column

    IF df has multi-level column headers THEN
        FOR each column in df:
            IF first level of column header (col[0]) is not empty:
                USE col[0] as column name
            ELSE:
                USE col[1] as column name

    DISPLAY first few rows of df
END
```

	Date	Close	High	Low	Open	Volume
0	2014-01-02	27.646961	27.761958	27.526251	27.705080	72783144
1	2014-01-03	27.445286	27.741595	27.443548	27.693659	66601332
2	2014-01-06	27.751278	27.789527	27.481048	27.644230	70701228
3	2014-01-07	28.286280	28.306895	27.846658	27.942033	102001896
4	2014-01-08	28.345140	28.496400	28.147932	28.463615	89610300

Figure 1: Initial data

Handling Missing Values

*During data collection, some rows may have **missing values** due to gaps in market data or API issues. These rows can interfere with model training and lead to inaccurate predictions. To ensure the dataset is consistent and reliable, all rows containing missing values are removed using `dropna()`. After removal, `isnull().sum()` is used to verify that no missing values remain.*

```
Close     0
High      0
Low       0
Open      0
Volume    0
MA20      0
MA50      0
MA100     0
RSI       0
dtype: int64
```

Datatransformation

Data transformation includes tasks such as resetting indices, adjusting prices, and flattening multi-level column headers.

The columns **High, Low, Open, Volume,** and Date were deliberately removed because they do not directly contribute to the **sequential predictive modeling** of next-day closing prices in a Long Short-Term Memory network. Retaining only relevant features reduces **dimensionality**, mitigates **redundant information**, and decreases the risk of **overfitting**, thereby allowing the model to focus on the most informative signals.

Removing **Date** ensures the model only receives **numerical, sequential input**, since LSTM networks learn temporal dependencies from the sequence of data itself rather than explicit date labels.

```
df = df.drop(['Date'], axis = 1)

df=
df.drop(columns=['High','Low','Open',
'Volume'], axis=1)
```

In their place, we engineered **derived features** such as **moving averages over multiple temporal windows (20, 50, 100 days)** and the **Relative Strength Index (RSI)**. These features condense temporal price dynamics and momentum into **quantitative metrics**, enabling the LSTM to capture **short-term fluctuations, medium-term trends, and long-term dependencies** in the stock data, which enhances the model's ability to identify complex sequential patterns.

We included moving averages over **20, 50, and 100 days** to capture **price trends at multiple temporal scales**.

- **20-day moving average:** Reflects short-term price trends and fluctuations, allowing the model to detect **recent**

momentum and immediate market reactions.

- **50-day moving average:** Captures medium-term trends, smoothing out short-term volatility while retaining significant trend patterns.
- **100-day moving average:** Highlights nlong-term price movements, enabling the model to identify **sustained trends and macro-level market behavior**.

By combining these three windows, the LSTM network receives a **multi-scale representation of the stock's historical dynamics**, improving its ability to recognize **complex temporal dependencies** and anticipate next-day price movements more accurately.

The general formula to calculate moving averages is

$$\boldsymbol{MA_n(t)} = \frac{\boldsymbol{C_t + C_{t-1} + C_{t-2} + \cdots + C_{t-(n-1)}}}{\boldsymbol{n}} = \frac{\boldsymbol{1}}{\boldsymbol{n}}\sum_{\boldsymbol{i=0}}^{\boldsymbol{n-1}} \boldsymbol{C_{t-i}}$$

RSI calculation can be divided into four simple steps:

i. $Average\ Gain = \frac{\sum_{i=1}^{n} \max(C_{t-i+1} - C_{t-i}, 0)}{n}$

ii. $Average\ Loss = \frac{\sum_{i=1}^{n} \max(C_{t-i} - C_{t-i+1}, 0)}{n}$

iii. $RS = \frac{Average\ Gain}{Average\ Loss}$

iv. $RSI = 100 - \frac{100}{1+RS}$

RSI quantifies **momentum**, showing overbought (>70) or oversold (<30) conditions.

The rolling average smooths gains and losses over the period *n*.

Their Coding implementation is fairly straight forward

```
df['MA20'] = df['Close'].rolling(window=20).mean()
df['MA50'] = df['Close'].rolling(window=50).mean()
df['MA100'] = df['Close'].rolling(window=100).mean()
```

```
BEGIN
  FOR each time step t in the dataset starting from day 100:
      CALCULATE MA20 as the mean of Close prices from (t − 19) to t
      STORE the value in column MA20

      CALCULATE MA50 as the mean of Close prices from (t − 49) to t
      STORE the value in column MA50

      CALCULATE MA100 as the mean of Close prices from (t − 99) to t
      STORE the value in column MA100
  END FOR
END
```

```
def compute_rsi(series, period=14):
    delta = series.diff()

    gain = delta.where(delta > 0, 0)
    loss = -delta.where(delta < 0, 0)

    avg_gain = gain.rolling(period).mean()
    avg_loss = loss.rolling(period).mean()

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))

    return rsi
```

```
BEGIN
  INPUT closing_price_series, period = 14

  COMPUTE delta = difference between consecutive values in closing_price_series

  FOR each value in delta:
     IF delta > 0 THEN
        gain = delta
        loss = 0
     ELSE
        gain = 0
        loss = −delta
     END IF
  END FOR

  COMPUTE avg_gain = rolling mean of gain over 14 periods
  COMPUTE avg_loss = rolling mean of loss over 14 periods

  COMPUTE RS = avg_gain / avg_loss
  COMPUTE RSI = 100 − (100 / (1 + RS))

  RETURN RSI
END
```

The **14-day period** is the standard recommended by **J. Welles Wilder** [1]

It represents roughly **two trading weeks** of daily stock data, balancing responsiveness and stability:

- **Shorter periods (<14 days):** RSI reacts too quickly, generating more false signals and noise.
- **Longer periods (>14 days):** RSI becomes slow, lagging behind actual market momentum.

Using 14 days allows the RSI to capture **meaningful momentum trends** while filtering out minor price fluctuations, making it suitable for **daily stock prediction models**.

CombiningData

Combining datasets is the process of horizontally concatenating multiple feature matrices—such as raw OHLCV data and engineered technical indicators—into a single unified dataset. This step aligns all features temporally by the date index, ensuring consistent observation windows for sequential modeling. It also resolves discrepancies in feature dimensions caused by rolling-window computations (moving averages and RSI). After merging, the dataset is verified for integrity and any rows with missing values are removed to maintain a complete time series for training the Long Short-Term Memory network.

The final dataset is represented as a feature matrix **X**, where each row corresponds to a trading day and each column represents a selected feature. In our implementation, the feature set includes the Close price, Moving Averages (20-day, 50-day, 100-day), and the Relative Strength Index, i.e.,

$$\boldsymbol{X} = [\boldsymbol{Close},\ \boldsymbol{MA_{20}},\ \boldsymbol{MA_{50}},\ \boldsymbol{MA_{100}},\ \boldsymbol{RSI}]$$

Close	MA20	MA50	MA100	RSI
63.039650	59.043561	60.257284	61.454919	68.185178
63.069439	59.309702	60.153410	61.542308	74.480013
63.387222	59.510798	60.046953	61.634365	78.999171
62.860901	59.617552	59.948839	61.694942	74.754902
62.642426	59.747644	59.871578	61.746184	66.464013

Figure 2:Final feature matrix

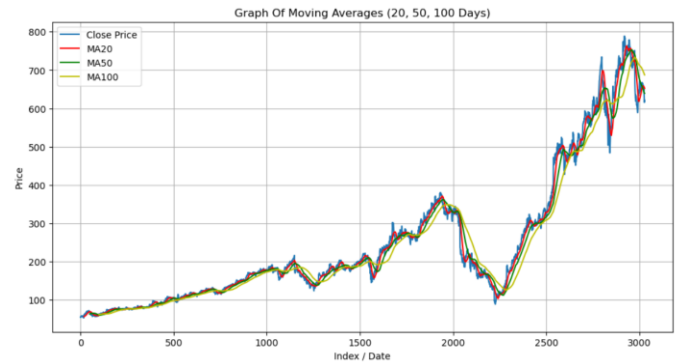


Figure 3: Close price with moving averages

Feature Scaling

Feature scaling is a preprocessing step used to standardize the range of independent variables before feeding them into a neural network. In our dataset, different features have different numeric ranges; for example, the Close price can be in hundreds of dollars, while the Relative Strength Index is always between 0 and 100. Such scale differences can lead to biased learning, where features with larger values dominate the training process and slow down convergence.

To address this, we used **Min-Max scaling**, which transforms all features into a uniform range [0,1][0, 1][0,1]. This normalization ensures that each feature contributes proportionately during training and improves the stability of gradient-based optimization. Min-Max scaling is particularly suitable for LSTM models because it preserves the relative temporal patterns and the distribution shape of the original data, which is essential for time-series forecasting.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0,1))
```

```
scaler.fit(train)
```

```
data_training_array = scaler.transform(train)
```

The formula for scaling is

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

The scaler is fitted on the training set and then applied to both training and testing datasets to maintain consistency and prevent data leakage. This approach helps the model to learn efficiently and generalize better on unseen data.

Sequence Generation for LSTM Input

Long Short-Term Memory networks require input data in sequential form, where each training example contains a fixed window of past observations. Therefore, the time-series dataset must be transformed into overlapping sequences. This process converts the tabular data into a three-dimensional structure suitable for LSTM training.

The following code generates input sequences (x_train) and their corresponding targets (y_train):

```
x_train = []
y_train = []

for i in range(100, data_training_array.shape[0]):
    x_train.append(data_training_array[i-100: i])
    y_train.append(data_training_array[i, 0])

x_train, y_train = np.array(x_train), np.array(y_train)
```

The code initializes two lists: **x_train** for storing input sequences and **y_train** for storing target values. The loop starts at index 100 because the first training sample requires 100 previous days of data. For each index **i**, the previous 100 rows are extracted as the input sequence, while the target is set to the Close price of the current day **(data_training_array[i, 0]).** After processing all available data, both lists are converted into **NumPy** arrays, resulting in an input shape of (samples, 100, features) for the **LSTM model.**

Assume the Close prices for the first 101 days are:

Index	Close
0	100
1	102
...	...
99	110
100	112

After running the code, the first training sample will be:

- **Input (`x_train[0]`):** Close prices from index 0 to 99
- **Target (`y_train[0]`):** Close price at index 100 → **112**

This means the model learns to predict the next day's closing price using the previous 100 days of data.

1. BEGIN
2. INITIALIZE empty list x_train
3. INITIALIZE empty list y_train
4.
5. FOR each index i from 100 to end of training data:
6. EXTRACT previous 100 rows from data_training_array (from i-100 to i-1)
7. APPEND extracted sequence to x_train
8.
9. SELECT target value as data_training_array[i, 0] (Close price)
10. APPEND target value to y_train
11.
12. CONVERT x_train to NumPy array

Before making predictions on the test set, the LSTM model requires a continuous sequence of past data to generate the first input window. The following code combines the last 100 days of training data with the test dataset, creating a continuous time-series sequence:

```
past_100_days = pd.DataFrame(train[-100:])
```

```
test_df = pd.DataFrame(test)
```

```
# Combine them
final_df = pd.concat([past_100_days, test_df], ignore_index=True)

final_df.head()
```

```
input_data = scaler.transform(final_df)
input_data
```

- **past_100_days** extracts the last 100 rows from the training dataset. These rows represent the most recent historical observations prior to the test period.
- **test_df** converts the test dataset into a DataFrame format.
- **pd.concat()** merges the two DataFrames vertically, ensuring that the test data follows

immediately after the last training observation.

- **The resulting final_df** forms a continuous time series, which is essential because the first prediction in the test set requires the previous 100 days of data as input.
- Using **ignore_index=True** resets the index so the combined DataFrame has a clean sequential index from 0 to n.

Data Exploration

During the exploration phase, we first computed descriptive statistics for the Close price and engineered features. The Close price demonstrated high variance and skewness, indicating significant volatility and the presence of outliers. The moving averages exhibited reduced variance, confirming their role in smoothing short-term fluctuations and highlighting long-term trend components.

We then performed time-series visualization to inspect trend and noise behavior. The Close price showed multiple regime shifts with abrupt trend reversals, validating the need for a model capable of learning long-term dependencies. The 20-day moving average responded rapidly to price changes, whereas the 100-day moving average captured slower, persistent trends. The Relative Strength Index displayed oscillatory behavior within the bounded range of 0–100, and periods where RSI exceeded 70 or fell below 30 corresponded to momentum extremes and subsequent price corrections.

Correlation analysis was conducted to assess feature redundancy and linear dependency. The moving averages were highly correlated with the Close price and with each other, indicating multicollinearity among trend-based features. However, the Relative Strength Index showed weaker correlation with moving averages, suggesting it provides orthogonal momentum information. These findings confirmed that the selected feature set is informative while maintaining diversity in the signal space, which supports effective sequential learning by the LSTM model.

Data Modelling

Model Architecture and Variable Definition

The model is defined using a sequential neural network architecture, which is appropriate for time-series forecasting. We used Long Short-Term Memory (LSTM) layers because they can capture long-term dependencies and sequential patterns in stock price data. The model consists of two LSTM layers followed by a Dense output layer.

```
# from keras.models import Sequential
# from keras.layers import Input, LSTM, Dense, Dropout
# from keras.callbacks import EarlyStopping


# model = Sequential()
# model.add(Input(shape=(x_train.shape[1], x_train.shape[2])))
# model.add(LSTM(units=96, return_sequences=True))
# model.add(Dropout(0.03))


# model.add(LSTM(units=48))
# model.add(Dense(units=1))
# model.compile(optimizer='adam', loss='mean_squared_error')
```

Dropout is included to reduce overfitting.

`x_train`: Input sequences with shape (samples, 100, features)

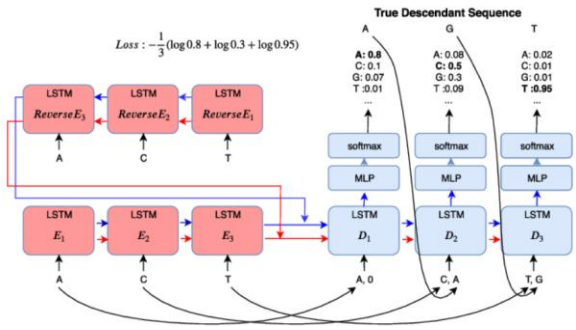
`y_train`: Target values (next day Close price)

`units=96` and `units=48`: Number of LSTM neurons in each layer

`Dropout(0.03)`: Regularization to prevent overfitting

`Dense(units=1)`: Single output neuron for predicting next-day price

Model Sequence



The proposed model employs a **Sequential Long Short-Term Memory (LSTM)** architecture to capture temporal dependencies in historical stock data. The input layer is configured to accept sequences of length **100 time steps**, where each time step consists of multiple features such as closing price, moving averages, and technical indicators. This structure enables the model to learn from extended historical patterns rather than isolated data points.

The first LSTM layer contains **96 units** and is configured with return_sequences=True, allowing it to output a full sequence rather than a single vector. This enables subsequent recurrent layers to receive temporally enriched representations. A **dropout layer (0.03)** follows, serving as a regularization mechanism to mitigate overfitting by randomly deactivating a fraction of neurons during training.

The second LSTM layer comprises **48 units** and outputs a single vector representation. This layer synthesizes the sequential information into a compact feature vector that encapsulates the learned temporal dynamics. Finally, a **dense layer with a single neuron** produces the predicted closing price for the next time step.

The model is compiled using the **Adam optimizer**, chosen for its adaptive learning rate capabilities and robust performance on noisy datasets such as financial time series. The **mean squared error (MSE)** loss function is utilized to quantify prediction accuracy, as it penalizes larger deviations more heavily, which is critical for financial forecasting tasks.

Early stopping is used to stop training when validation loss stops improving, which prevents overfitting and reduces training time.

```
#   early_stopping = EarlyStopping
#   (
#   monitor='val_loss',
#   patience=20,
#   restore_best_weights=True
#   )


#   history = model.fit
#   (
#   x_train,
#   y_train,
#   epochs=60,
#   batch_size=32,
#   validation_data=(x_test, y_test),
#   callbacks=[early_stopping],
#   verbose=1
#   )
```

`epochs=60`: Maximum number of training iterations

`batch_size=32`: Number of samples processed before weight update

`validation_data`: Used to evaluate model performance during training

Model Diagnostics

Model diagnostics are used to assess training behavior and validate the model's learning process. We analyze the training and validation loss curves to check for overfitting or underfitting. If validation loss increases while training loss decreases, it indicates overfitting. Early stopping helps mitigate this issue by restoring the best weights based on validation performance.

Diagnostics include:

Training loss vs validation loss curves

Final training and validation loss values

Observing whether loss stabilizes or diverges

Inverse Scaling

After the model predicts the **scaled** closing prices (`y_pred`), the output values are still in the **normalized range (0–1)** because the data was previously scaled using `MinMaxScaler`. To interpret the predictions in real-world terms (actual stock prices), the predictions must be converted back to their original scale.

However, the scaler was fitted on **multiple features** (e.g., Close, MA20, MA50, MA100, RSI). The scaler expects an input array with the same number of features during inverse transformation. Since the prediction only contains the **Close price**, we must reconstruct a dummy array with the same number of features to satisfy the scaler.

Presentation and Automation

```
# y_pred is shape (N,1)


# Step 1: Create dummy array with 5 features

dummy_pred = np.zeros((len(y_pred), 5))


# Step 2: Put predicted Close values in column 0

dummy_pred[:, 0] = y_pred[:, 0]


# Step 3: Inverse transform

y_pred_actual = scaler.inverse_transform(dummy_pred)[:, 0]
```

For each stock, the system reports the predicted next-day closing price alongside historical closing prices, enabling direct comparison between actual and predicted values. Visual representation is particularly emphasized, as time-series plots allow patterns, trends,

```
1.  BEGIN
2.  CREATE dummy_pred ∈ ℝ^(N×5) filled with zeros
3.  SET dummy_pred[:, 0] = y_pred[:, 0]
4.  APPLY inverse scaling:
5.  y_pred_actual_full = scaler.inverse_transform(dummy_pred)
6.  EXTRACT Close column:
7.  y_pred_actual = y_pred_actual_full[:, 0]
8.  END
```

and deviations between model predictions and real market behavior to be easily observed. This dual presentation approach improves interpretability and supports qualitative evaluation of the model's forecasting capability over time.

```
#   plt.figure(figsize = (12,6))
#   plt.plot(y_test, 'b', label = "Original Price")
#   plt.plot(y_pred, 'r', label = "Predicted Price")
#   plt.xlabel('Time')
#   plt.ylabel('Price')
#   plt.legend()
#   plt.grid(True)
#   plt.show()
```

Figure 4: Actual vs Predicted Close Prices for Selected Stock

Actual_Close	Predicted_Close
87.158859	88.510674
88.618790	87.811311
86.791397	87.567349
85.430771	87.429636
87.844131	87.000677

Figure 5: Actual vs Predicted Close prices

Model Performance Evaluation

To assess the predictive performance of the proposed Long Short-Term Memory model, quantitative evaluation metrics were employed on the unseen test dataset. Since stock price prediction is a regression task, error-based and goodness-of-fit metrics were selected to evaluate both prediction accuracy and explanatory power.

The Mean Absolute Error was computed and normalized by the mean of the actual test values to express error as a percentage, allowing scale-independent interpretation across different stock prices. This metric captures the average deviation between predicted and actual closing prices.

In addition, the coefficient of determination (R^2 score) was used to measure how well the model explains variance in the observed data. The R^2 score provides insight into the alignment between predicted and actual price movements, with higher values indicating stronger model fit. To support interpretability, the R^2 score was also visualized using a horizontal bar chart.

```
# from sklearn.metrics import mean_absolute_error
# mae = mean_absolute_error(y_test, y_pred)
# mae_percentage = (mae / np.mean(y_test)) * 100
# print("Mean absolute error on test set: {:.2f}%".format(mae_percentage))
```

```
Mean absolute error on test set: 3.08%
```

Figure 7: Mean Absolute Error Percentage(MAEP)

```
# # Plotting the R2 score
# fig, ax = plt.subplots()
# ax.barh(0, r2, color='skyblue')
# ax.set_xlim([-1, 1])
# ax.set_yticks([])
# ax.set_xlabel('R2 Score')
# ax.set_title('R2 Score')
# # Adding the R2 score value on the bar
# ax.text(r2, 0, f'{r2:.2f}', va='center', color='black')
# plt.show()
```

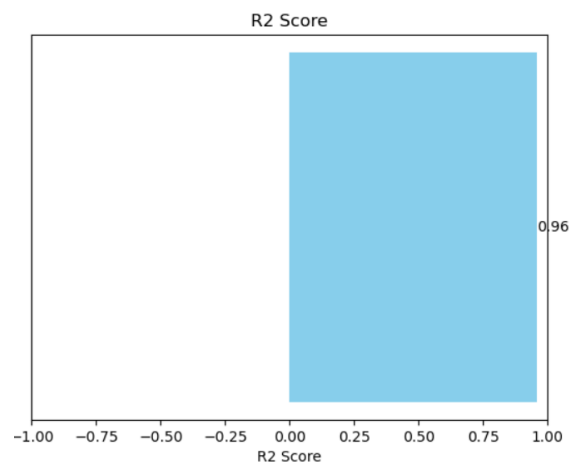


Figure 6: R2 Score

```
# plt.scatter(actual, predicted)
# plt.plot([min(actual), max(actual)], [min(predicted), max(predicted)], 'r--')
# plt.xlabel('Actual Values')
# plt.ylabel('Predicted Values')
# plt.title(f'R2 Score: {r2:.2f}')
# plt.show()
```

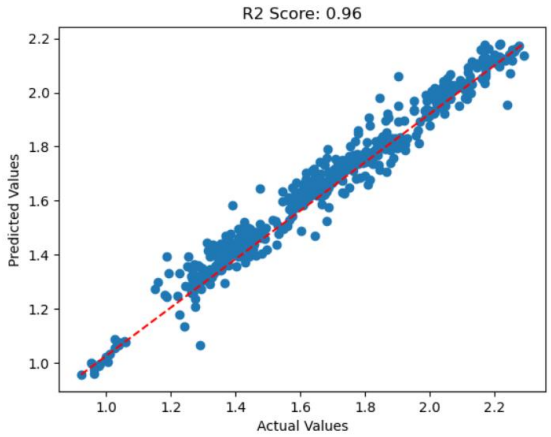


Figure 8:Actual vs Predicted

LimitationAnd Challenges

Despite achieving meaningful results, the project is subject to several limitations. Stock market data is inherently volatile and influenced by external factors such as news events, economic conditions, and market sentiment, which are not captured by historical price-based features alone. As a result, the model's predictions are constrained by the information available in past price movements and technical indicators. Additionally, the system focuses on next-day price prediction, which limits its applicability for longer-term forecasting. The generated decision-support signals are therefore intended strictly for analytical demonstration and should not be interpreted as reliable financial guidance.

Throughout the development process, multiple practical challenges were encountered. Acquiring consistent and reliable historical data required careful selection of data sources. Feature selection and scaling demanded iterative experimentation to ensure stable model behavior and effective learning. Designing an appropriate Long Short-Term Memory architecture involved tuning the number of layers, hidden units, and sequence length to balance model complexity and generalization. Selecting suitable hyperparameters, particularly dropout values, was challenging, as improper choices could result in underfitting or overfitting. Furthermore, the choice of loss function and the use of early stopping to restore the best-performing weights were critical decisions to prevent unnecessary training and improve model robustness.

Appendix A

Model Training and Optimization

The model is compiled using the Adam optimizer and Mean Squared Error as the loss function, which is suitable for regression tasks such as stock price prediction. Early stopping is applied to prevent overfitting, where training is halted if the validation loss does not improve for 20 consecutive epochs. The best model weights are restored automatically when training stops. The model is then trained for a maximum of 60 epochs with a batch size of 32, using the validation set to monitor generalization performance. The training process returns a history object that records training and validation loss values for each epoch, enabling further analysis of model convergence.

Appendix B

System Design and Automation

The system is designed as an end-to-end pipeline that integrates data retrieval, preprocessing, model inference, and result visualization. Real-time market data is fetched automatically using the Yahoo Finance API, after which preprocessing steps such as feature engineering and scaling are applied. The appropriate pre-trained model and corresponding scaler are then loaded dynamically based on the selected stock ticker. Predictions are generated in real time and returned to the frontend through a REST API. The user interface displays the prediction results along with graphical visualizations and decision-support signals. This automated design ensures that the system operates without manual intervention, improving reproducibility and demonstrating practical deployment considerations for deep learning applications in financial analytics.

References

[1] J. W. Wilder, "MEASURING INTERNAL STRENGTH: WILDER'S RSI INDICATOR," 1978.